Feedback Final Project

ELEC 4475

7/22/2025

Matt Zubricki, Ryan Arbid, Brett Tata

Summary of the Design Problem

The objective of this project is to design a feedback velocity control system for a DC motor using the QUBE Servo motor. The system must control the motor to reach a desired angular velocity of 12 rad/s while maintaining zero steady-state error. In addition to achieving the desired velocity, the system must satisfy transient response requirements of a percentage overshoot of 9% or less and a settling time of 0.7 seconds or less. The voltage-to-velocity transfer function developed in Lab #4 is used as the mathematical model of the DC motor.

Compensators are added to the feedback control system to meet these requirements. A PI compensator is used to eliminate steady-state error, while a lead compensator is used to improve the transient response by reducing settling time and controlling overshoot. The completed controller is first evaluated through simulation to verify its performance before being implemented experimentally on the QUBE Servo motor. The simulated and experimental responses are then evaluated to determine whether the final design satisfies all specified performance requirements.

Design

We want to have our system meet the design requirements provided to us. The system we may need to compensate using controllers is the following:

$$G_U(s) = \frac{8670.13}{(s + 8.389)(s + 50)}$$

This was the equation for the model of our DC motor we found in Lab #4. This is the base system we will be working on for this project.

The first part of the design process is to find the design point. The design point of our system is the location on the root locus where our system requirements will be fulfilled. These include the percentage overshoot and settling time mentioned previously. The design point also needs to sit on the damping ratio line. If we can make the root locus of our final design pass through this design point, we will have completed our goal.

The design point we are looking for is the dominant pole of our system that we are trying to get. This point is a complex pole that has a real part σ_d and a complex part of $j \cdot \omega_d$. Finding these two parts is our first objective.

$$Design\ Point\ =\ -\sigma_d + j \cdot \omega_d$$

To start the design process, we first had to find the damping ratio, since our design point needs to be on the damping ratio line. To find the damping ratio, we can use our desired percentage overshoot to solve it. Essentially, if we want to have a ≤9% percentage overshoot, we need to find the damping ratio associated with that overshoot. This can be done by taking the overshoot equation and reconstructing it to have ζ on one side. ζ is now an expression in terms of OS%.

$$OS = e^{-\left(\frac{\pi\zeta}{\sqrt{1-\zeta^2}}\right)}$$

(1. Overshoot Equation)

$$\zeta = \sqrt{\frac{\ln^2\left(\frac{OS\%}{100}\right)}{\ln^2\left(\frac{OS\%}{100}\right)+\pi^2}}$$

(2. Rearranged OS Equation for ζ)

When we plug in 9% into the equation for ζ , we get 0.608. This damping ratio will be the core of our design, as we always want to operate on this damping ratio line.

Now that we have the damping ratio, we need to find the actual design point that our system will want to operate at. To achieve this, we want to use other design requirements we have, the settling time of $\leq 0.7s$. We can use this settling time to find the real part of our complex pole, σ_d. To do this efficiently, we'll have to approximate our system to be a 2nd-order system. If that assumption is made, we can find σ_d with this equation:

With a desired settling time of 0.7s, we get a $\sigma_d = 5.714$

Next, we want to find the complex part of our design point, ω_d. To find ω_d, we can use the σ_d we just found, and the damping ratio from earlier. We can use those values in this equation here:

$$\omega_d = \sigma_d \tan(\arccos(\zeta))$$

After plugging in σ_d and ζ values, $\omega_d = 7.461$.

By putting these values together, we can get our design point.

$$Design\,point\ =\ 5.714\ +\ j7.461$$

With the help of MATLAB, we can plot the root locus of our system and check whether we need to add any compensators. When checking MATLAB, we find that our system intersects with the damping ratio line at $-29.3 + j38.3$. This point is the dominant pole location of our system, and we want the dominant pole location of our system to be the same as our design point. These values do not match, however, so we must start designing compensators to fix it.

We have a couple of design requirements to meet with our original system that will require compensators to fix. One's the transient response of the system, and the other is achieving a zero steady-state error. These two parameters cannot be achieved with one compensator; we must use two compensators for our system.

We had to choose either to start with eliminating the zero-state error by calculating the PI compensator equation or to fix the transient response by calculating the lead compensator design. We chose to start with the PI design, meaning we had to find the gain of the equation and a zero value.

$$G_{PI}(s) = \frac{K_P(s + Z_P)}{s}$$

When designing a PI compensator, you can choose a zero value yourself, as the steady-state error will become zero due to the pure integrator, s, in the denominator of the PI compensator equation. Though we couldn't pick any value for Z_P, we needed to pick one close to the origin, but not too close to the origin, since we could experience some pole-zero cancellation with Z_P and our pure integrator. For our design, we chose a Z_P value of 6. For PI compensators, the gain, K_P, is usually approximately 1, so we went with 1 for our gain. Our PI compensator equation is the following:

$$G_{PI}(s) = \frac{(s + 6)}{s}$$

After creating our PI compensator, we now have a new system to evaluate:

$$G_{PC}(s) = \frac{K(s + 6)}{s(s + 8.389)(s + 50)}$$

Our next step is to fix the transient properties of the system since the PI compensator only affected the steady-state error. To achieve this, we designed a lead compensator. The general equation for a lead compensator is the following:

$$G_{Lead}(s) = \frac{K_c(s + Z_c)}{(s + P_c)}$$

We have a zero and a pole we need to find for this compensator to work. Luckily, though, we can assume one of their values and then solve for the other one with the one we assumed. It is common practice to assume the zero value. We didn't arbitrarily pick a value though; we wanted the zero value to be the left-most point on the real axis. This is because with the additional pole, it would push the root locus closer to the right, and the design point is further to the right than our current dominant pole location. With that being said, we chose a Z_c value of 60.

To find the location of the pole Pc, we found the angles between each real open-loop pole or zero and the design point. If we can find all these angles besides the unknown pole angle, we can input them into this expression to solve that missing angle, since this expression must be true for our root locus to work.

$$\Sigma(OL\ Zeros)\ -\ \Sigma(OL\ Poles) = \pm 180°$$

With that missing angle, we can find that poles location with a simple trigonometric expression:

$$\tan(\theta_{Pc}) = \frac{7.461}{(P_c - 5.714)},\ P_c = 8.675$$

The final part of the lead compensator that we need to find is the gain, K_c. To find K_c, we found the original gain of the system, $K_1 = 8670.13$, and the gain of the fully compensated system, $K_2 = 65.9$, which was found in MATLAB by checking the intercept between the root locus and the design point. With these gains, we can solve the expression $K_c = \frac{K_2}{K_1} = 0.0076$. With that gain, we know have the completed lead compensator:

$$G_{Lead}(s) = \frac{0.0076(s + 60)}{(s + 8.675)}$$

Bringing all the compensators and the original system together, we get our final compensated system:

$$G_{(s)} = \frac{65.9(s + 6)(s + 60)}{s(s + 8.389)(s + 8.675)(s + 50)}$$

We can validate this system with our requirements using the simulation results in MATLAB, which can be seen in the next section.

Simulation Results

These simulations show us the performance of each system and what parts of those systems still need help. We will show the root locus of each system and its corresponding step response to show how each system would be simulated. Each system is run at the total gain of the system, with the gain of each compensator accounted for. Our original system from Lab #4 runs at a gain of 8670.13, so that is the gain we start at.

The first system to show is the original system, $G_u(s)$. As expected during the design portion, the system does not satisfy our design requirements. The root locus of $G_u(s)$ doesn't intersect with the damping ratio line at the design point. We can see why this is important by looking at the step response.

The step response for $G_u(s)$ shows us that our system has an overshoot percentage of 36.2% and a settling time of 0.117s when the system is run at a gain of 8670.13. The system also settles at a value of 0.952, which means the system has a steady-state error of 4.8%. None of our design requirements were met, except for the settling time. Even though the settling time is less than 0.7s, it is far below the settling time we designed around.

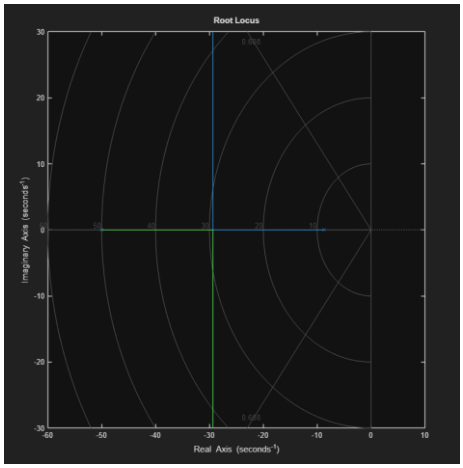


Figure 1-Root Locus of original system, $G_u(s)$

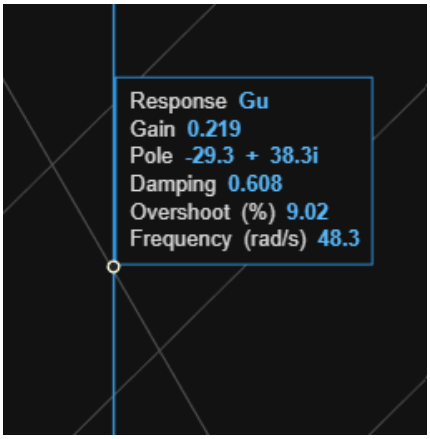


Figure 2-Root Locus Intersection with damping ratio line. K=8670.13

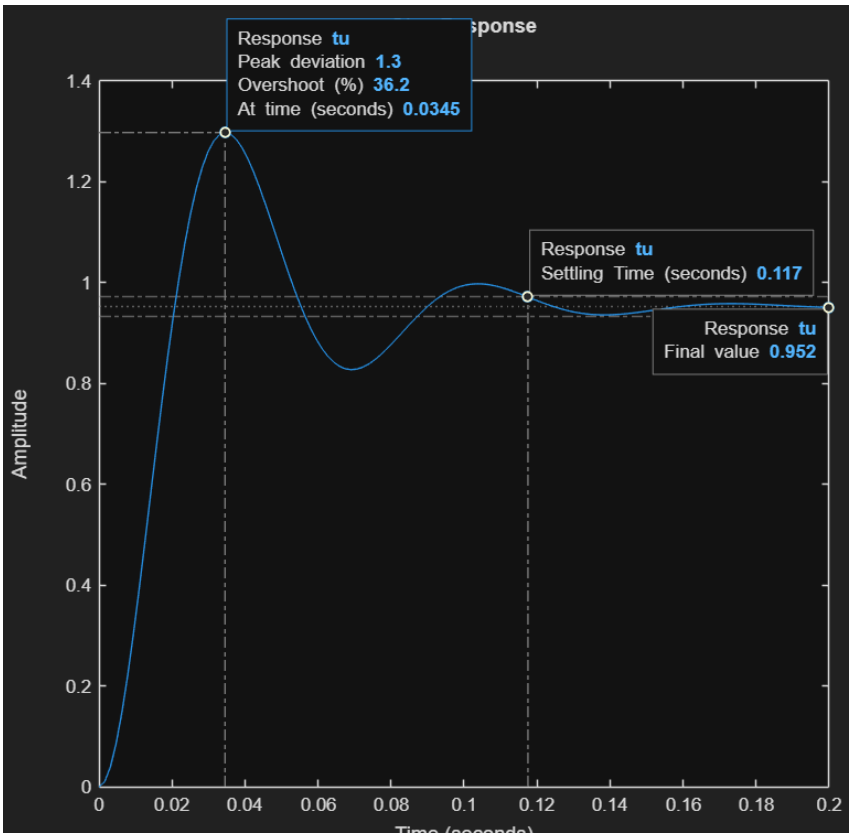


Figure 3-Step response of original system, K = 8670.13

The next system to show is the system, $G_p(s)$. This is the system that has the original system and a PI compensator in cascade. The root locus for $G_p(s)$ looks almost identical to the original system's root locus, except the breakaway point was shifted to the right slightly.

The step response for $G_p(s)$ shows us that our system has an overshoot percentage of 38.1% and a settling time of 0.151s when the system is run at a gain of 8670.13. This system does, however, have a steady-state error of zero, thanks to the PI compensator.

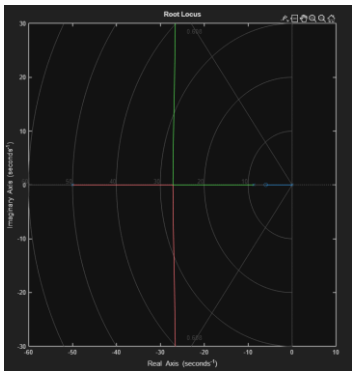


Figure 4-Root Locus of original system with a PI compensator, $G_p(s)$

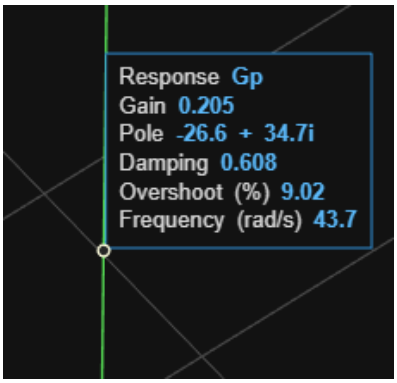


Figure 5-Root Locus Intersection with damping ratio line. K = 8670.13

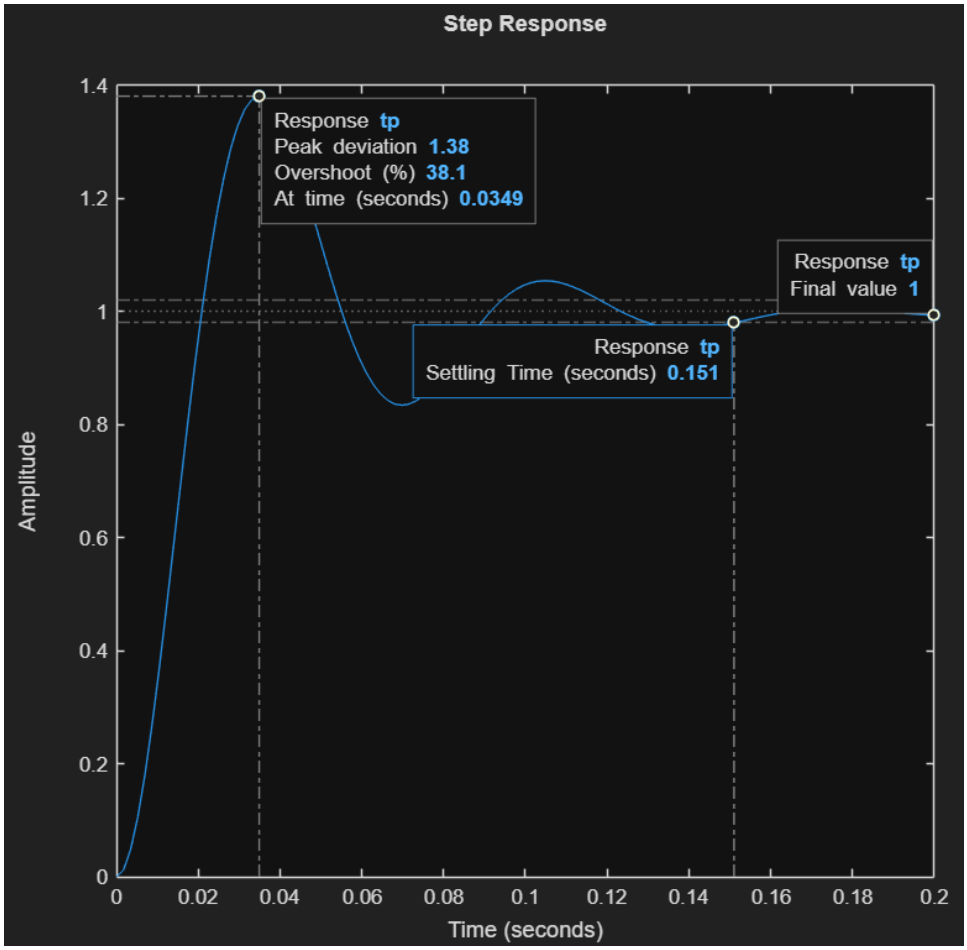


Figure 6-Step response of new system $G_p(s)$, K = 8670.13

The system to show is the system, $G_c(s)$. This is the system that has the original system, a PI compensator, and a lead compensator in cascade. The root locus for $G_c(s)$ now intersects with the damping ratio line at the design point found earlier. This system is run with a gain of 65.9 since it is a combination of the original system's gain and the gains of the two compensators.

The step response for $G_c(s)$ shows us that our system has an overshoot percentage of 5.75% and a settling time of 0.612s when the system is run at a gain of 65.9. This system also has a steady-state error of zero. All these parameters match our design requirements.

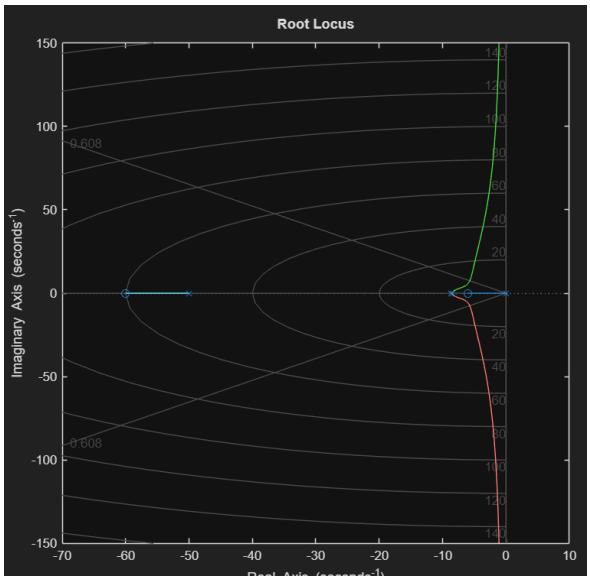


Figure 7-Root Locus of compensated system, $G_c(s)$

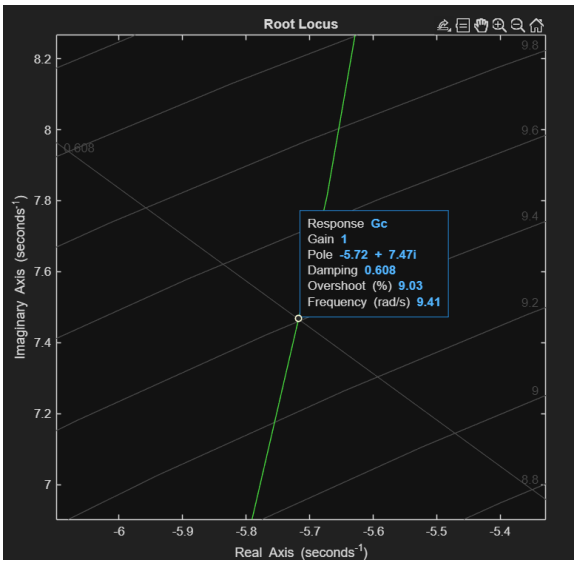


Figure 8-Root Locus Intersection with damping ratio line. K = 65.9

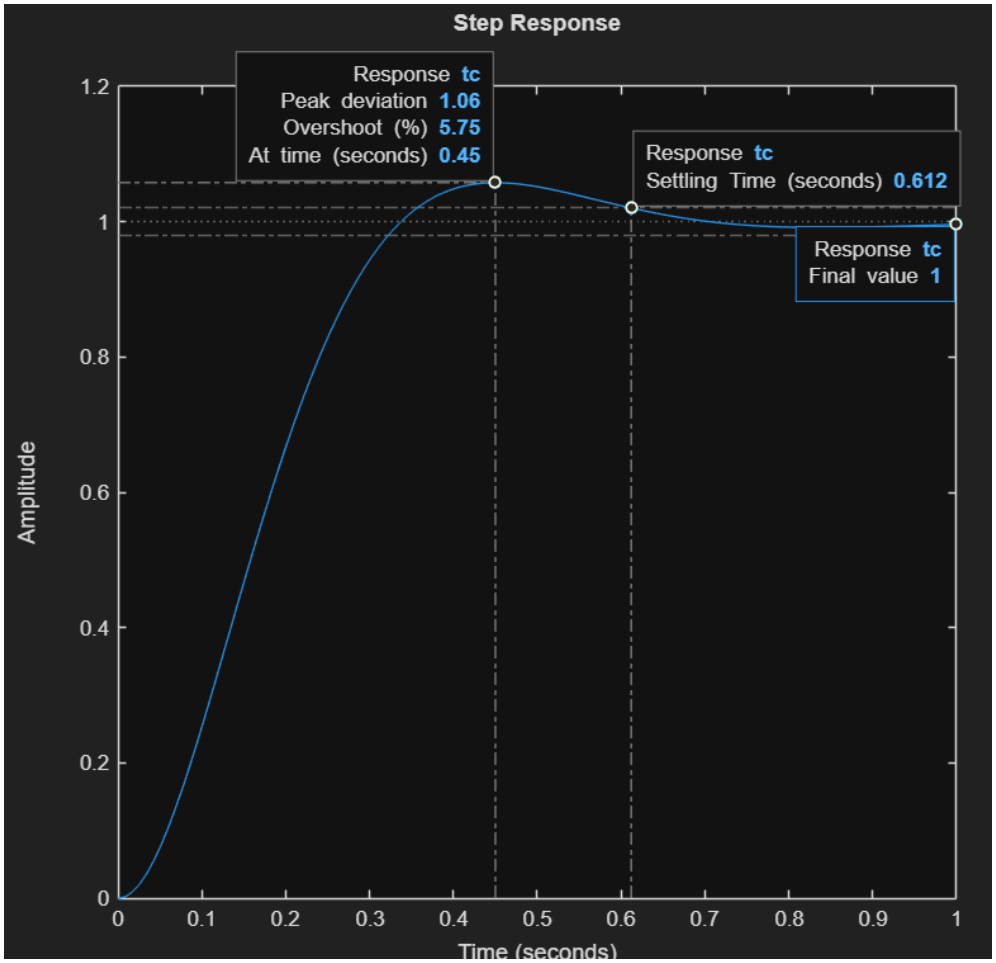


Figure 9-Step response of compensated system, $G_c(s)$, K = 65.9

When comparing the compensated system with the uncompensated system, we can clearly see the differences in transient response and steady state. We can see the improvement in the steady-state error by the difference between the settling values. We can see the correction in the overshoot percentage and the increase in settling time, which is closer to 0.7, as part of our design.

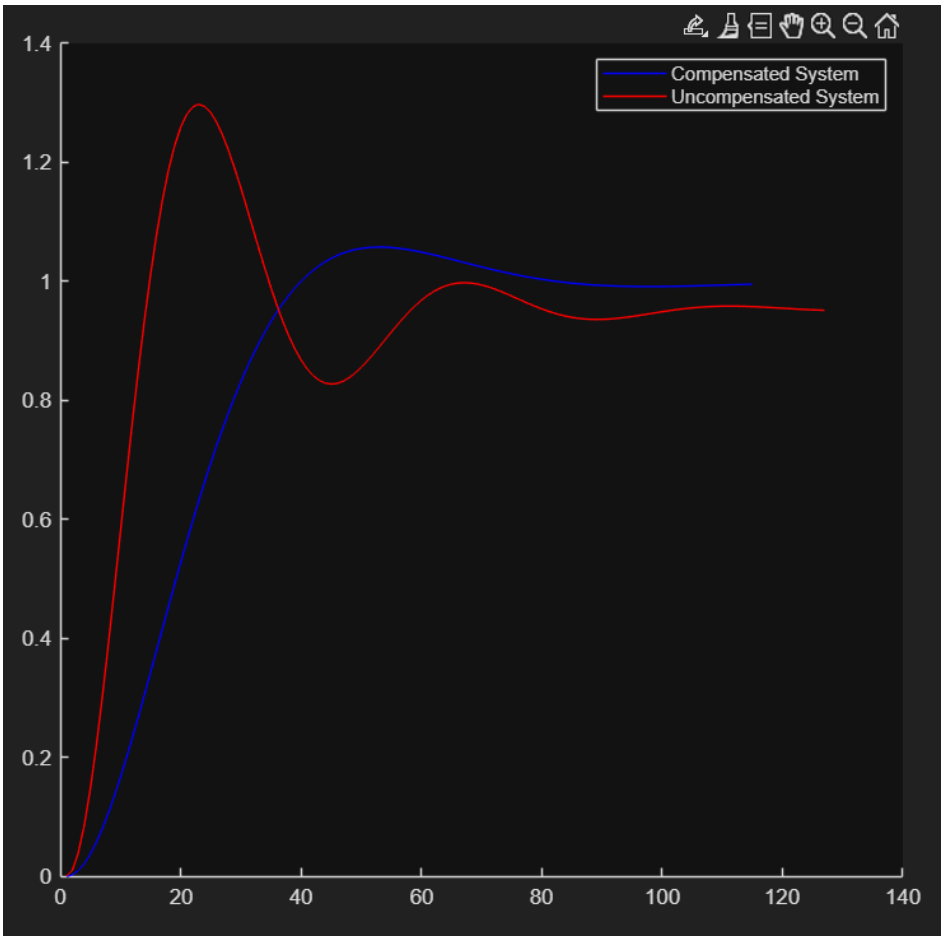


Figure 10-Uncompensated system and Compensated system

Disturbance:

To test our compensated system, $G_c(s)$, we compared the step response of $G_c(s)$, with a step response of a negative feedback loop with $G_c(s)$ and a step input disturbance of 1 rad/sec placed somewhere between $R(s)$ and $C(s)$. This is looking to see what effects any random disturbances could have on the compensators or the overall system. We'll be checking to see if the transient properties or the steady-state error will be affected.

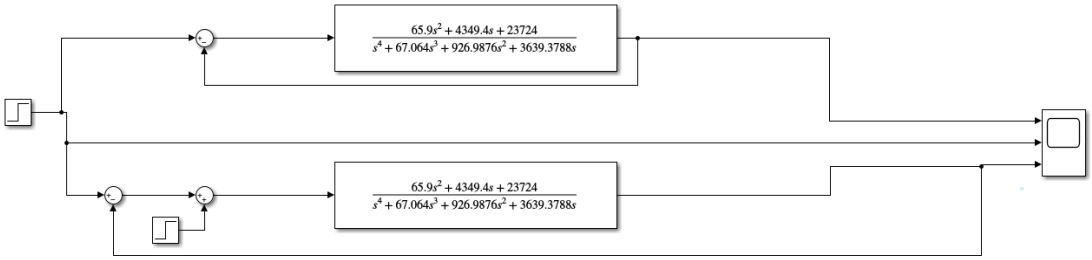


Figure 11-Compensated system and Compensated system with disturbance, model in Simulink

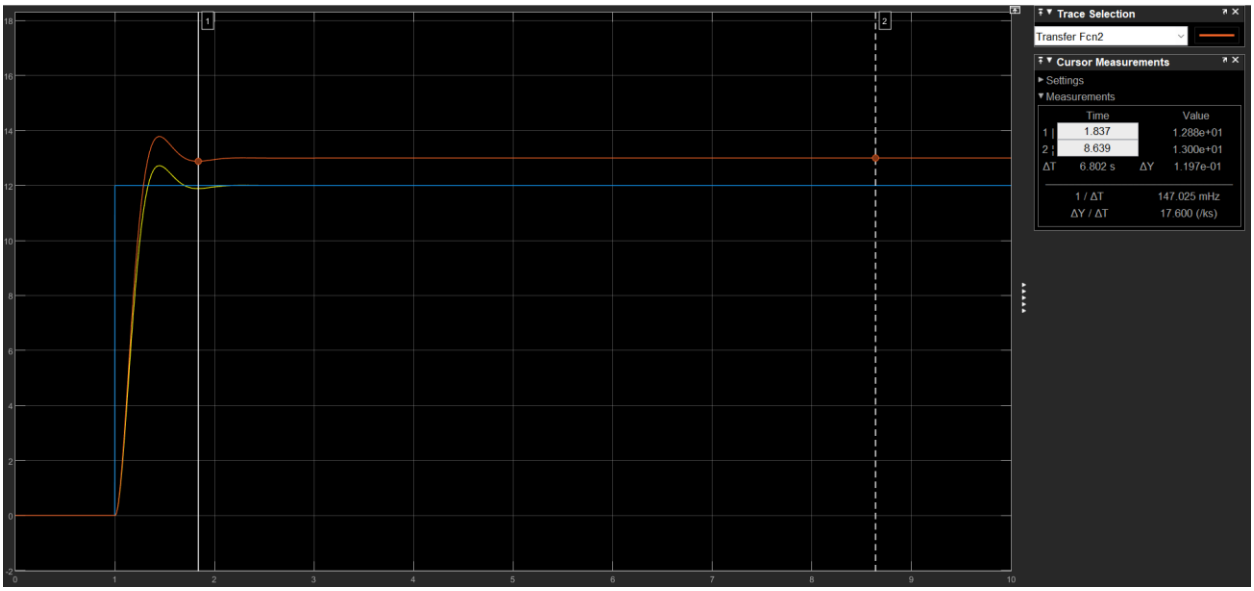


Figure 12-Compensated system and Compensated system with disturbance, step response

Experimental Results

The final control system was implemented on the QUBE Servo motor using the Simulink model shown in Figure 1. A step input of 12rad/s was applied to the system. The designed PI and lead compensators were implemented in the feedback loop to satisfy the steady-state and transient-response requirements. The QUBE encoder measured the motor position, which was converted into angular velocity and fed back into the control system.

During experimental testing, an additional filter was added to the system because the original experimental response contained excessive noise and did not provide a sufficiently stable velocity signal. The filter reduced the effects of encoder and measurement noise and allowed the physical motor to follow the desired response. This additional filtering was necessary for the experimental system because the mathematical model does not fully account for the noise and physical disturbances present in the actual QUBE Servo.

The experimental velocity response is shown in Figure 2. The motor quickly approached the desired 12rad/s reference and reached a maximum velocity of approximately 12.7rad/s, equating to a percentage overshoot of 5.8%. This satisfies the requirement of %OS $\leq$ 9%. The measured settling time was 0.616s, which also satisfies the requirement of $T_s \leq 0.7$s. After settling, the motor remained at 12 rad/s, demonstrating zero steady-state error. Small fluctuations in the response were observed due to measurement of noise and the physical behavior of the motor. Overall, the experimental results demonstrate that the final control system satisfied the required steady-state velocity, percentage overshoot, and settling time.

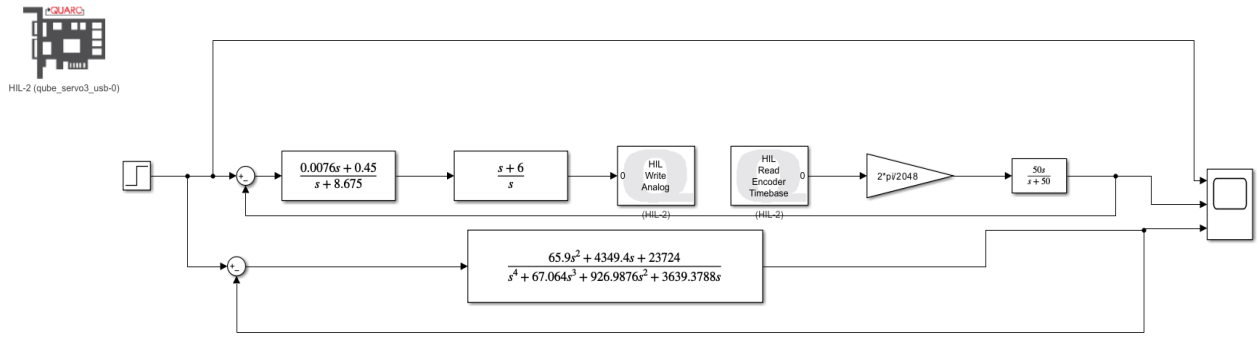


Figure 13-Simulink Diagram

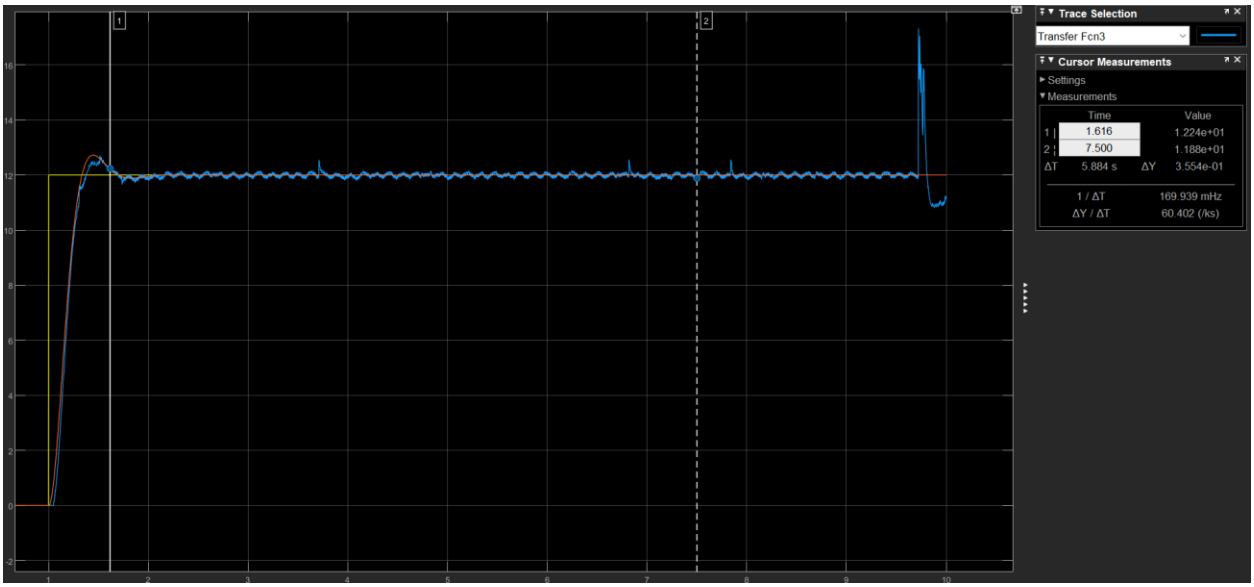


Figure 14-Simulink Results

Conclusions

In this project, we successfully designed a feedback velocity control system for a DC motor that was able to reach the target angular velocity of 12rad/s while meeting the required transient response specifications of %OS ≤ 9% and Ts ≤ 0.7 s. We used a cascaded compensator design that included a PI compensator to eliminate steady-state error and a lead compensator to improve the transient response. After testing and adjusting the controller, the final system achieved zero steady-state error, a percentage overshoot of 5.8%, and a settling time of 0.616s.

During the hardware portion of the project, we also added a low-pass filter to reduce noise in the measured velocity signal and make the response more stable. This was an important addition because the physical system did not behave exactly like the ideal simulation due to noise, disturbances, and other real-world effects. Testing both the simulation and the actual motor helped us see how the controller performed under more realistic conditions and showed why some adjustments were necessary when moving from the mathematical model to the hardware. Overall, the final system met the required performance specifications and gave us a better understanding of how PI and lead compensation can be used together to improve the performance of a real feedback control system.